House Price Prediction

End-to-End Machine Learning Application with Streamlit

Devixo Solutions Internship – Task 04

Submitted by: **Wajiha Waqar**

BS Robotics and Intelligent Systems, NUML Islamabad

Date: 31 October 2026

Resource	Link
GitHub repository	https://github.com/wajihawaqar903-cpu/Devixo-Solutions-Task-04

Contents

- 1. Introduction
- 2. Dataset Description
- 3. Data Preprocessing
- 4. Exploratory Data Analysis
- 5. Model Training and Comparison
- 6. Hyperparameter Tuning
- 7. Final Model Selection and Saving
- 8. Streamlit Application
- 9. Deployment
- 10. Conclusion and Future Work
- Appendix: Project Structure and How to Run

1. Introduction

The objective of this task was to build a complete, end-to-end machine learning application: select a real-world problem, prepare the data, train and compare several models, tune the best one, and deliver it to users through a web application. This report documents the full workflow for a **House Price Prediction** system, in which a user enters the details of a house and receives an estimated price together with supporting information about that estimate.

Because the target variable (Price) is a continuous number, the problem is a **regression** problem. Models were therefore evaluated with regression metrics (MAE, MSE, RMSE and R^2) rather than classification metrics such as accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC, which do not apply here.

Tools used: Python, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, Joblib, Streamlit, Google Colab and GitHub.

2. Dataset Description

The dataset is a house price dataset (file: *house price prediction dataset.csv*) containing **2,000 records** and **10 columns**. The target variable is **Price**.

Column	Type	Description
Id	Integer	Unique record identifier (not used for training)
Area	Numeric	Size of the house
Bedrooms	Numeric	Number of bedrooms
Bathrooms	Numeric	Number of bathrooms
Floors	Numeric	Number of floors
YearBuilt	Numeric	Year the house was built
Location	Categorical	Location of the house
Condition	Categorical	Condition of the house
Garage	Categorical	Garage availability
Price	Numeric	**Target variable**: price of the house

3. Data Preprocessing

3.1 Loading and inspection

The dataset was uploaded to Google Colab and loaded with Pandas. Its shape, data types and summary statistics were inspected using *info()* and *describe()*.

3.2 Missing values

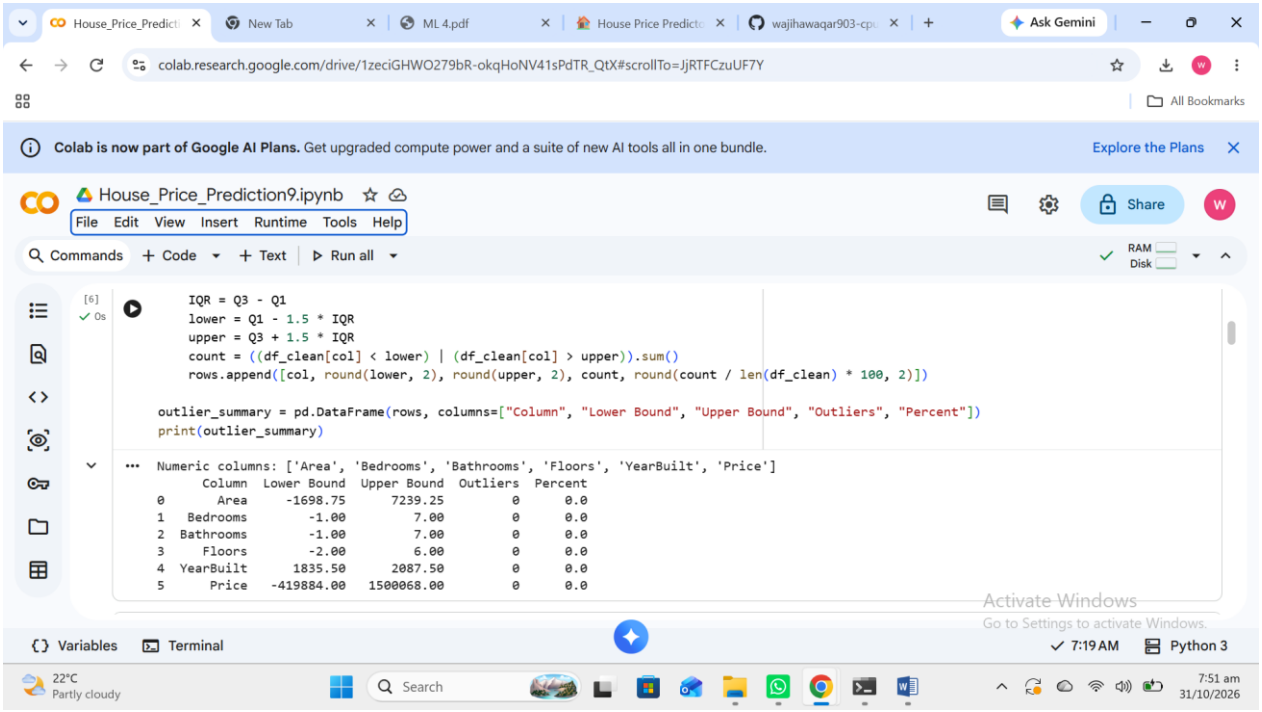
The dataset was checked for missing values in every column. **No missing values were found**, so no imputation was required.

3.3 Duplicate records

Duplicate rows were checked both on all columns and with the Id column ignored (since Id is unique and could hide repeated records). Duplicates found:0. Rows remaining after removal 0.

3.4 Outlier detection

Outliers in the numeric columns were detected with the Interquartile Range (IQR) method. A value was flagged if it fell below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR. Boxplots were used to visualise them



3.5 Feature engineering

New features were created from the existing columns to give the models more information. No feature was derived from the target (for example price per unit area), because that would leak the answer into the model.

New feature	Formula	Purpose
HouseAge	2026 − YearBuilt	Age is easier for a model to use than a calendar year
TotalRooms	Bedrooms + Bathrooms	Overall size in rooms
AreaPerBedroom	Area / (Bedrooms + 1)	Spaciousness of rooms
BathBedRatio	Bathrooms / (Bedrooms + 1)	Comfort / layout indicator
AreaPerFloor	Area / Floors	Footprint of the building
IsNew	1 if HouseAge ≤ 10, else 0	Flags recently built houses
AgeGroup	New / Modern / Old / Very Old	Categorical age band

The original YearBuilt column was dropped afterwards, because HouseAge carries the same information.

3.6 Encoding categorical variables

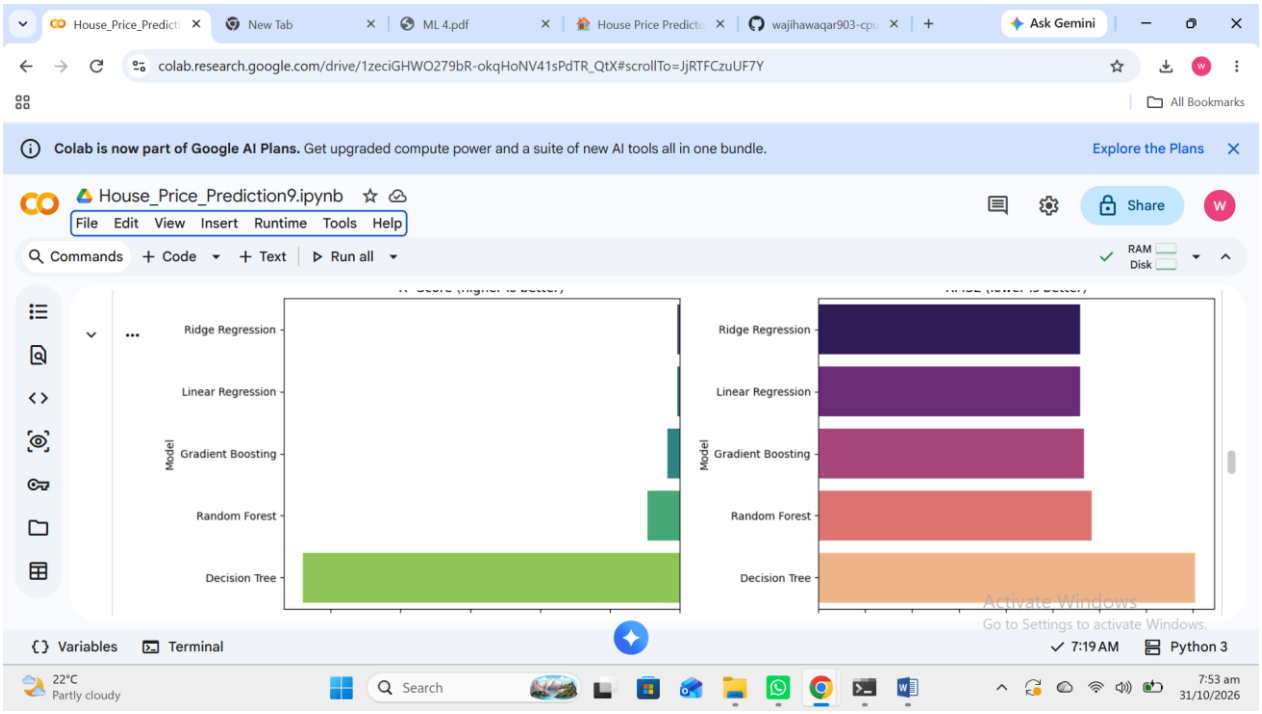
The text columns (Location, Condition, Garage and the new AgeGroup) were converted to numbers with **one-hot encoding** (*pandas.get_dummies* with *drop_first=True* to avoid redundant columns). The Id column was removed before training because it is only a row label.

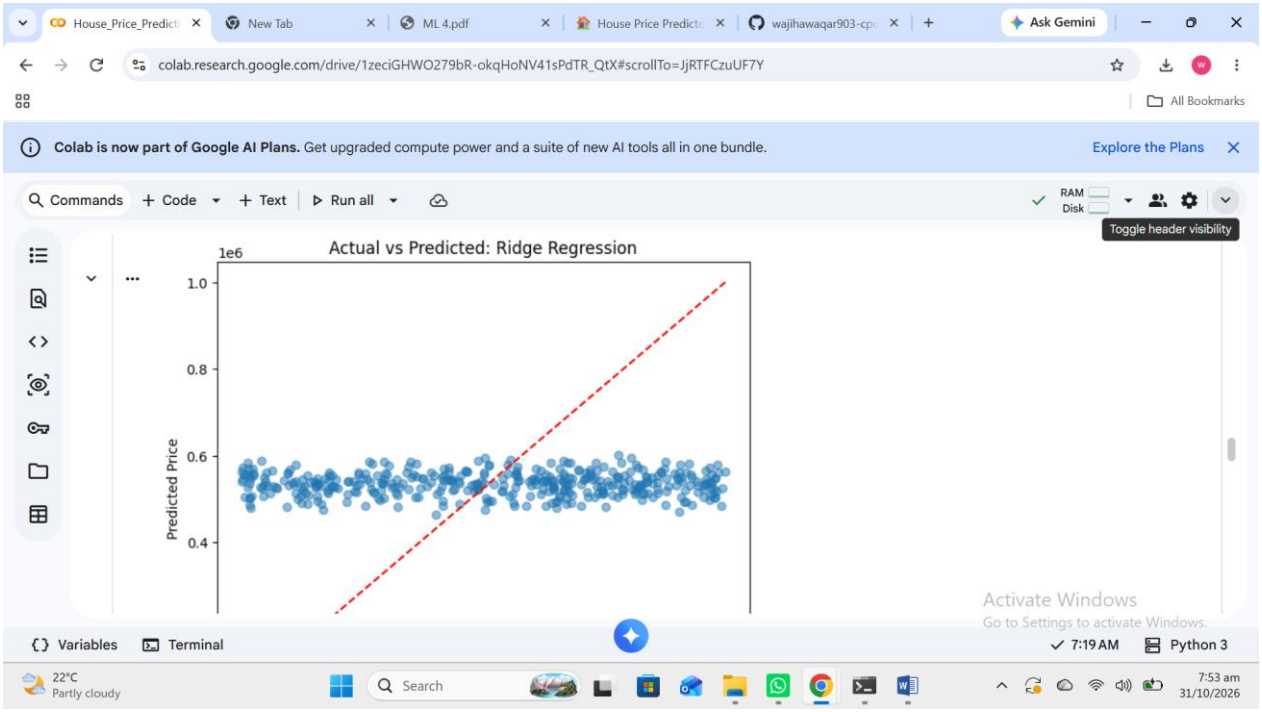
3.7 Train/test split and scaling

The data was split into **80% training** and **20% testing** sets (random_state = 42). Numeric features were then standardised with **StandardScaler**, which was **fitted on the training set only** and applied to the test set, so that no information from the test data leaked into training. The one-hot columns and the target Price were not scaled. The fitted scaler was saved so that the web application can apply exactly the same transformation to user input.

4. Exploratory Data Analysis

Charts were produced to understand the data before modelling.





5. Model Training and Comparison

Five regression models were trained on the training set and evaluated on the unseen test set:

- Linear Regression
- Ridge Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor

5.1 Evaluation metrics

Metric	Meaning
MAE	Mean Absolute Error: average size of the prediction error, in the same units as Price
MSE	Mean Squared Error: average of squared errors; penalises large mistakes more
RMSE	Root Mean Squared Error: square root of MSE, in the same units as Price
R^2 Score	Share of the variation in Price explained by the model (1.0 is perfect)

5.3 Overfitting check

5-fold cross-validation was run on the training data and the training R^2 was compared with the cross-validated R^2. A large gap between the two would indicate overfitting.

Figure 5: Comparison of model performance.

Best model before tuning: [model name] with R^2 = [value] and RMSE = [value].

6. Hyperparameter Tuning

To improve performance, two search methods from Scikit-learn were applied, both with 5-fold cross-validation and R^2 as the scoring metric:

- **RandomizedSearchCV** on the Random Forest (20 random combinations).
- **GridSearchCV** on the Gradient Boosting model (exhaustive search over a smaller grid).

Model	Parameters searched
Random Forest (RandomizedSearchCV)	n_estimators [100, 200, 300]; max_depth [None, 10, 20, 30]; min_samples_split [2, 5, 10]; min_samples_leaf [1, 2, 4]; max_features [sqrt, log2, 1.0]
Gradient Boosting (GridSearchCV)	n_estimators [100, 200]; learning_rate [0.05, 0.1]; max_depth [3, 4, 5]; subsample [0.8, 1.0]

7. Final Model Selection and Saving

All tuned and default models were compared on the test set, and the one with the highest R^2 was selected as the final model

The following files were saved with Joblib so that the application can reproduce the exact training pipeline:

- **best_model.pkl**: the trained final model
- **scaler.pkl**: the fitted StandardScaler
- **feature_columns.pkl**: the exact column order used in training
- **metadata.pkl**: dropdown options, input ranges, dataset price statistics and test metrics shown in the app

8. Streamlit Application

The final model was wrapped in an interactive web application built with Streamlit (*app.py*). The application satisfies the task requirements as follows:

Requirement	How the app meets it
Enter required information	A form with inputs for Area, Bedrooms, Bathrooms, Floors, Year Built, Location, Condition and Garage
Click a prediction button	The **Predict Price** button runs the prediction
Receive the prediction	The estimated price is shown prominently, with a typical error range based on the model's MAE
See relevant information about the prediction	Comparison with the dataset's average price, price percentile, price per unit area, the engineered features used, a feature-importance chart (when the model supports it), and model information

How a prediction is produced

- The user's input is converted into the same engineered features used in training (HouseAge, TotalRooms, etc.).
- Categorical values are one-hot encoded and arranged in the exact column order saved in feature_columns.pkl.
- The numeric columns are scaled with the saved scaler.
- The saved model predicts the price, and the surrounding information is computed from the saved metadata.

10. Conclusion and Future Work

This project covered the full machine learning lifecycle: data cleaning, feature engineering, model comparison, hyperparameter tuning, model saving, and delivery through a local web application. Predictions are estimates based on historical data and are not a professional valuation.

- The model only knows the features in the dataset; real prices also depend on factors such as exact neighbourhood, market conditions and property features that were not recorded.
- Inputs far outside the range of the training data may give unreliable results.

Future improvements

- Add more features and a larger, more recent dataset.
- Try additional models such as XGBoost or LightGBM.
- Add SHAP-based explanations for each individual prediction.
- Add confidence intervals and a feature to compare several houses side by side.

Appendix: Project Structure and How to Run

Repository structure

```
house-price-app/
  app.py
  requirements.txt
  best_model.pkl
  scaler.pkl
  feature_columns.pkl
  metadata.pkl
  README.md
  (notebook, dataset and screenshots folders)
```

Run locally

```
pip install -r requirements.txt
python -m streamlit run app.py
```